LAB 3: MQTT & ESP32

José Paca, *UTEC,* Daniel Casquino, *UTEC,* Nicolas Stigler, *UTEC,* Sofia Ku, *UTEC*

I. Introduction

THIS laboratory approaches the connection between data acquisition on an ESP32 and message distribution using MQTT. The guide organizes experiments in connectivity, telemetry, control and serial integration [1].

A. Theoretical Framework

MQTT uses the publisher–subscriber model: a client publishes to a topic, and the server distributes messages according to subscriptions. MQTT 3.1.1 requires an ordered, bidirectional transport, like TCP. Its QoS level 0 offers delivery at most once; it does not guarantee that every message will reach the receiver [2]. Therefore, simply observing a connection is not enough to quantify message loss or receipt of sensor readings.

In the Arduino-ESP32 API, `analogRead` returns a raw, uncalibrated ADC code, while `analogReadMilliVolts` provides a calibrated result in millivolts [3]. An ADC code obtained from a divider with an LDR does not directly equate to illuminance in lux. The relation with illumination requires characterizing the sensor, circuit, and reference conditions; this conversion is not performed here.

PubSubClient 2.8 selects MQTT 3.1.1 by default. Its overload `publish(topic,payload)` constructs a message without retention and without QoS bits 1/2; `loop()` handles client communication [4]. These properties describe the library and allow interpretation of the CP2 sketch; they do not replace a receiver capture.

B. State of the Art

MQTT is commonly studied as a lightweight application-layer protocol for resource-constrained IoT devices. Al-Fuqaha *et al.* identify publish–subscribe protocols, including MQTT, as an important part of the IoT protocol stack and relate protocol selection to device constraints, network conditions, and application requirements [5]. This perspective is relevant to the present experiment because the ESP32 acts as a constrained sensing client while the broker separates the publisher from the monitoring application.

Several studies compare MQTT with alternatives rather than evaluating it in isolation. Thangavel *et al.* evaluated MQTT and CoAP through a common middleware and showed that their behavior depends on the communication pattern and transport assumptions [6]. Similarly, the experimental study by Georgiou and Karapantelakis examined MQTT performance in IoT-to-cloud communication and reported that the selected QoS level affects latency and application behavior [7]. These results support documenting the protocol parameters instead of treating a successful connection as a complete performance evaluation.

For low-power wireless sensor networks, Hunkeler, Truong, and Stanford-Clark proposed MQTT-S as an adaptation of the MQTT model to networks where conventional TCP connectivity is not always available [8]. More recent work by Palmese, Redondi, and Cesana proposed adaptive QoS control for MQTT-S and evaluated its effect on delay and packet errors [9]. In contrast, this laboratory uses standard MQTT over Wi-Fi/TCP and QoS 0 through PubSubClient; therefore, the experiment demonstrates basic telemetry but does not claim the reliability or energy behavior of MQTT-S.

Recent measurements also investigate alternatives to TCP. Pacheco and Santos characterized delay and energy consumption for MQTT over TCP and QUIC using commercial off-the-shelf devices [10]. Their results reinforce that latency and energy depend on the complete transport stack and hardware, not only on the MQTT message format. The present work is consequently positioned as an implementation and functional validation of an ESP32–MQTT telemetry path. It provides evidence of acquisition and delivery, but it does not estimate lux, compare protocols statistically, or measure energy consumption.

II. Methodology

A. Components and CP2 configuration

The CP2 implementation uses an ESP32 development board, an LDR, a fixed resistor, a breadboard, jumper wires, and a USB cable. The LDR and the fixed resistor form a voltage divider. The divider output is connected to GPIO35, which is used as an ADC input by the ESP32. The exact resistance value and the orientation of the divider must match the physical circuit demonstrated during the checkpoint; changing either one changes the direction and scale of the ADC response.

The ESP32 is programmed with the Arduino IDE and the ESP32 board support package. The sketch uses the `WiFi.h` library for network connectivity and PubSubClient 2.8 for MQTT communication. The serial monitor is configured for 115200 baud. Network credentials are stored in `secrets_local.h`, which is intentionally excluded from the repository. The broker address is supplied through `mqtt_broker` and the broker listens on the standard non-TLS MQTT port, 1883. The client publishes the raw ADC value as a text payload to the `lab/ldr` topic.

The program first waits until the ESP32 is associated with the configured Wi-Fi network. It then configures the broker address and port. During each iteration, `analogRead(35)` acquires the raw ADC code, the value is printed to the serial monitor, and the MQTT connection is established if necessary. After `mqtt.loop()` services the client, the value is published and the program waits 1000 ms before the next

iteration. The 1000 ms delay is a minimum application delay rather than an exact sampling period because ADC acquisition, Wi-Fi processing, and MQTT operations add execution time.

B. Hardware and communication sequence

The complete data path is shown in Fig. 1. The sensor circuit produces an analog voltage, the ESP32 samples it, and the MQTT broker forwards the resulting payload to a subscribed client. Because `analogRead` returns an ADC code rather than a calibrated illuminance value, the reported payload is intentionally described as a raw LDR reading.

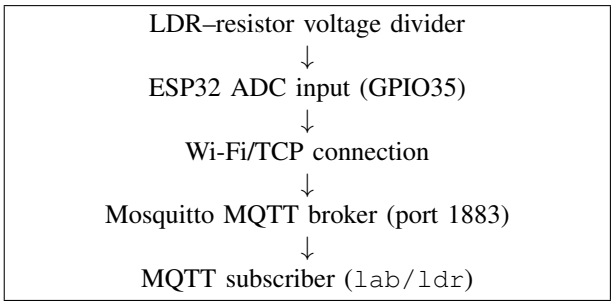


Figure 1. CP2 block diagram for LDR acquisition and MQTT telemetry.

Fig. 2 summarizes the execution sequence. A failed MQTT connection is reported through the serial monitor and causes the current iteration to return before the next attempt.

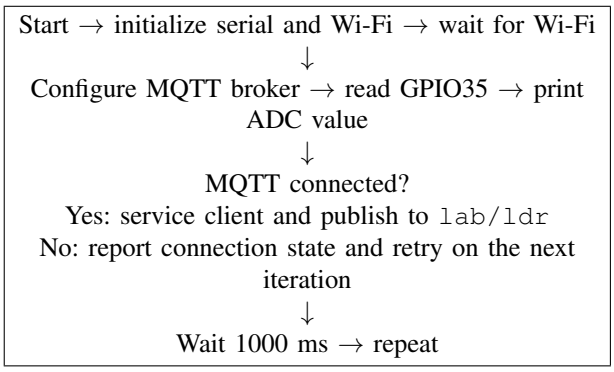


Figure 2. CP2 program flow for acquisition and MQTT publication.

III. Development

The purpose of this section is to answer both the checkpoint and report questions of the lab, as well as discuss any relevant results obtained during the experiences.

A. CHECKPOINT 1: Previous Installations

1) **Arduino Installation and ESP32 Setup**
First, we downloaded and installed the Arduino IDE for our computer's operating system, which in this case was Windows 11, as shown in Figure 3.
Once installed and opened, we needed to add support for the ESP32 board. To do this, we went to File > Preferences. There, we added the URL https://dl.espressif.com/dl/package_esp32_index.json in the "Additional Boards Manager URLs" field and clicked "OK", as shown in Figure 4.
Then, we needed to install the board support package for the ESP32. To do this, we went to Tools > Board > Boards Manager. There, we searched for "esp32" and installed the package provided by Espressif Systems, as shown in Figure 5.



Figure 3. Arduino IDE installation for Windows 11.

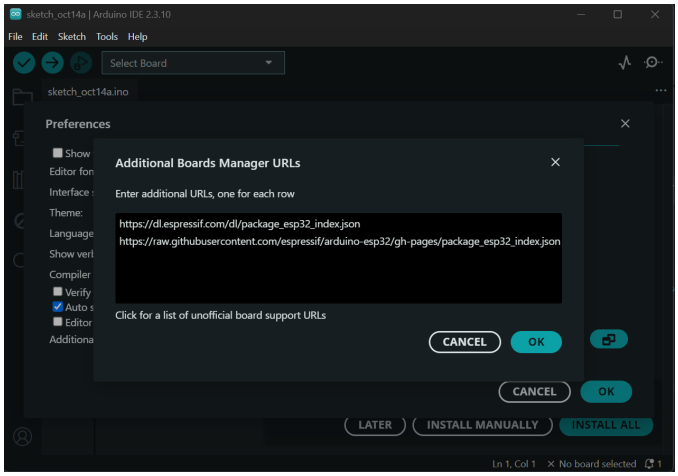


Figure 4. Arduino IDE preferences with the ESP32 board manager URL added.

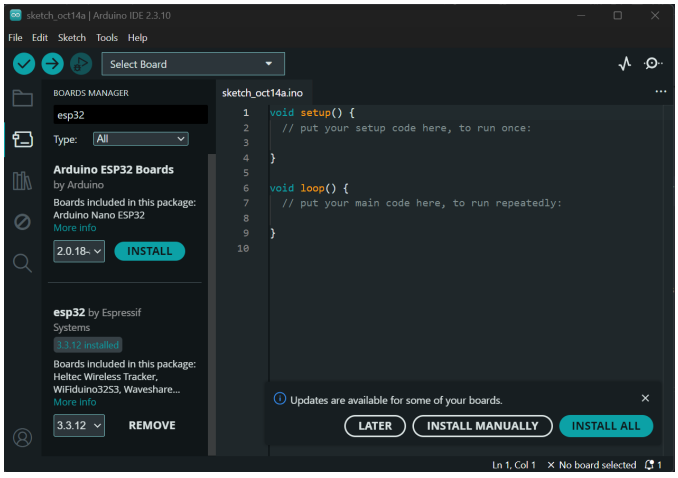


Figure 5. Installation of the ESP32 board support package by Espressif Systems.

To finish the initial setup, we went to Select Board > Select Other Board and Port. There, we searched for "NodeMCU-32S" in the board selection section and selected "COM5 Serial Port" in the ports section, as shown in Figure 6.

2) **Test Code Upload**
We tested a very simple code and actually ran into a

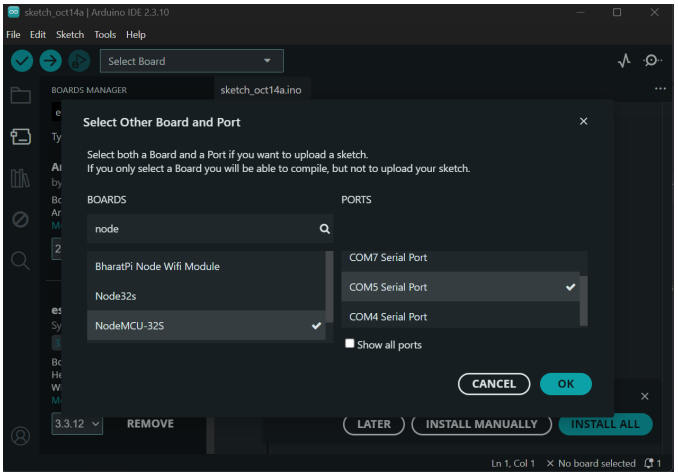


Figure 6. Initial selection of the NodeMCU-32S board and COM5 serial port in Arduino IDE.

small problem during the upload process. As shown in Figure 7, there was an issue with the selected serial port. We solved this by changing “COM5 Serial Port” to “COM3 Serial Port (USB)”, which corresponded to the connected ESP32, as shown in Figure 8. After fixing this issue, we successfully uploaded and ran the test code on the ESP32, as shown in Figures 9 and 10.

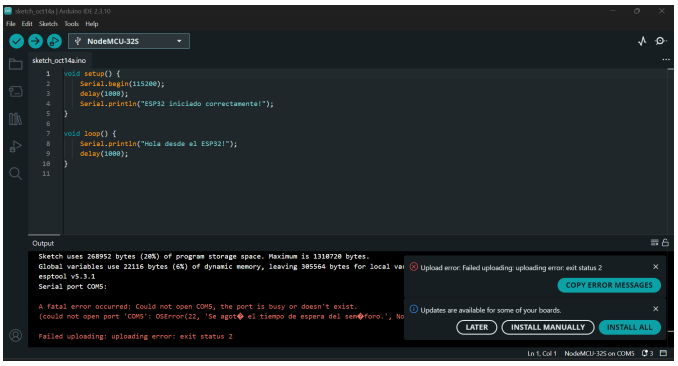


Figure 7. Error encountered during the initial code upload due to the incorrect serial port selection.

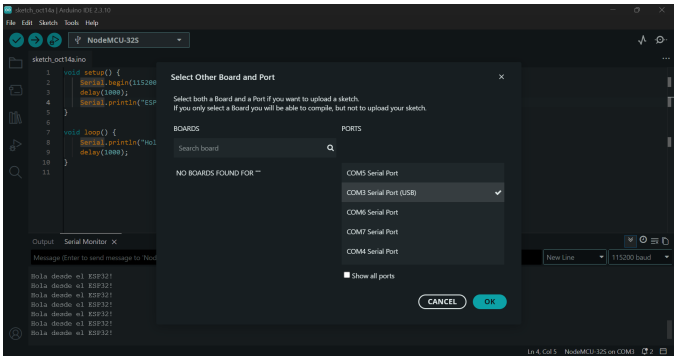


Figure 8. Correction of the serial port selection from COM5 to COM3 (USB).

3) Mosquitto Installation

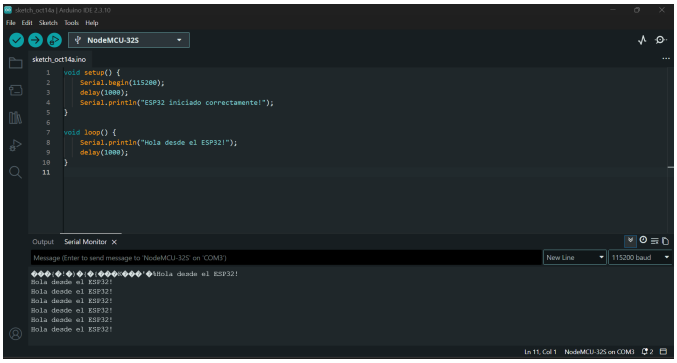


Figure 9. Successful compilation and upload of the test code to the ESP32.

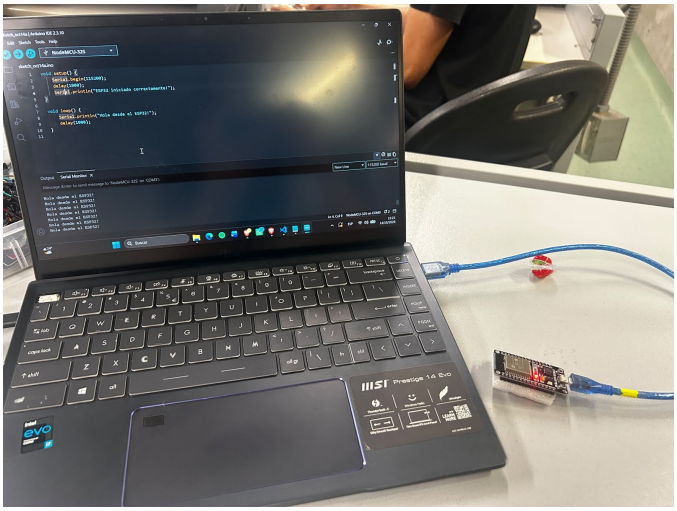


Figure 10. Arduino IDE Serial Monitor displaying the messages generated by the ESP32 test code.

First, we downloaded the Mosquitto executable installer for Windows, as shown in Figure 11. As instructed, Mosquitto was installed outside the Program Files directory. We also needed to add its installation path to the system environment variables in order to execute Mosquitto commands directly from the terminal. We then confirmed that Mosquitto could be executed successfully, as shown in Figure 12.

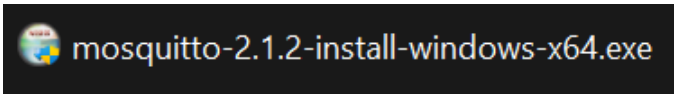


Figure 11. Download of the Mosquitto installer for Windows from its official website.

Then, we modified the Mosquitto configuration file, as shown in Figure 13, and successfully ran the broker using the updated configuration, as shown in Figure 14. Finally, we tested MQTT communication by subscribing to the topic “test/topic” and publishing a test message to the same topic. As shown in Figure 16, we published a message, and as shown in Figure 15, we were able to receive it successfully through the subscribed client.

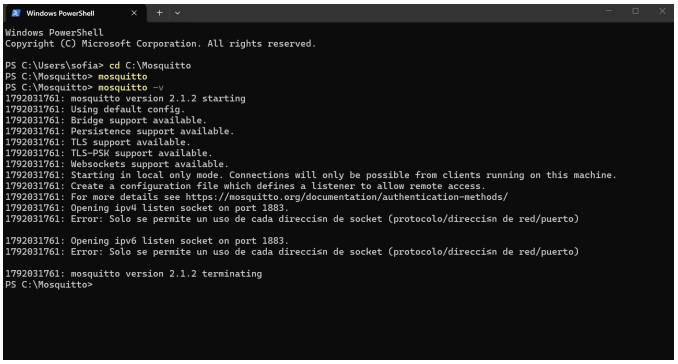


Figure 12. Verification of the Mosquitto installation through the Windows terminal.

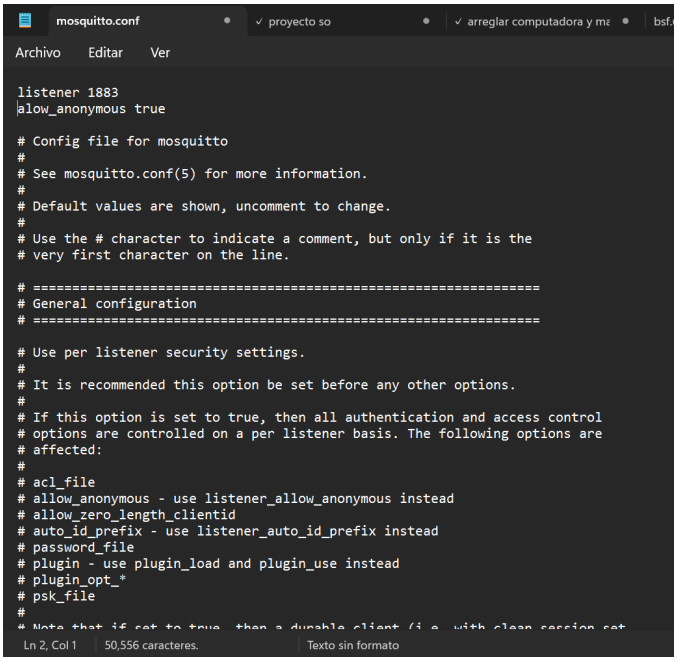


Figure 13. Modification of the Mosquitto configuration file to enable connections on port 1883 and allow anonymous clients.

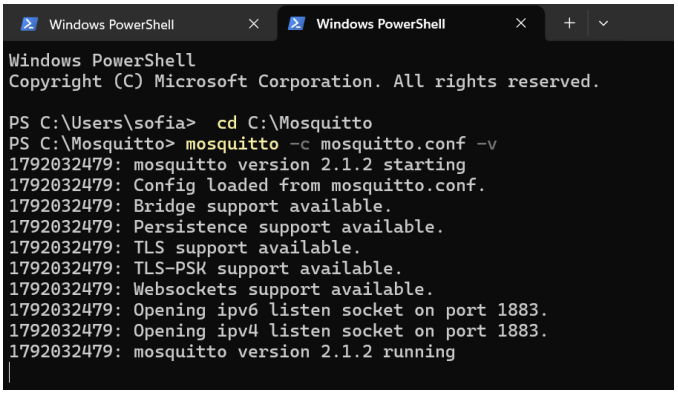


Figure 14. Successful execution of the Mosquitto MQTT broker using the updated configuration file.

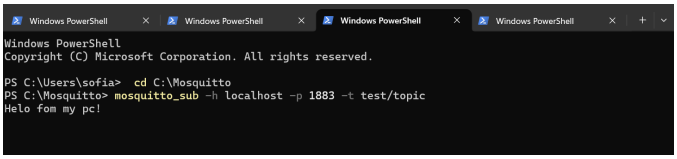


Figure 15. MQTT subscriber receiving the test message through the Mosquitto broker.

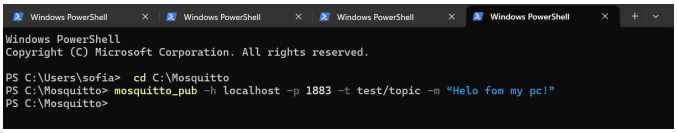


Figure 16. Publication of a test message to the MQTT topic using the Mosquitto command-line client.

B. CHECKPOINT 2: Basic Implementation with Reading

The second checkpoint implemented a complete telemetry path from an LDR connected to the ESP32 ADC to an MQTT subscriber. The circuit was assembled as a voltage divider and its output was connected to GPIO35. The ESP32 was programmed with the sketch stored in `checkpoint2/Checkpoint2_LDR/Checkpoint2_LDR.ino`. The credentials were kept in the local file `secrets_local.h`; no credentials were included in the shared source code.

1) Implementation procedure: The implementation followed these steps:

1) The LDR voltage-divider output was connected to GPIO35, while the circuit was supplied from the ESP32-compatible voltage rail and ground. The fixed-resistor value and the physical orientation were recorded from the assembled circuit because they determine the ADC response.
2) The ESP32 board support package and the PubSubClient library were enabled in Arduino IDE. The board and the serial port corresponding to the connected ESP32 were selected.
3) The local Wi-Fi SSID, password, and broker address were entered in `secrets_local.h`. The broker was configured on port 1883 and the subscriber was configured to listen to `lab/ldr`.
4) The sketch was compiled and uploaded to the ESP32. The serial monitor was opened at 115200 baud to verify Wi-Fi association, display the assigned IP address, and observe the ADC readings.
5) A Mosquitto subscriber was started before or during the test. The ESP32 then read the ADC, connected to the broker when necessary, and published each reading as a text payload.
6) The illumination over the LDR was changed while the serial output and the MQTT subscriber were observed. This test verifies that the acquisition path and the publication path respond to the same physical stimulus.

2) Checkpoint validation and technical discussion: The implementation is considered functionally validated when three observations occur together: the serial monitor displays

changing ADC codes, the broker accepts the ESP32 MQTT client, and the subscriber receives payloads on `lab/ldr`. The expected payload is an integer represented as text, because the sketch publishes `String(valor).c_str()`. The publication interval is approximately one second plus the time required by ADC, Wi-Fi, and MQTT operations; it should therefore not be reported as an exact one-second sampling period.

The reading is a raw ADC code and cannot be reported as illuminance in lux without a sensor calibration procedure. Increasing or decreasing the code with illumination also depends on which side of the divider contains the LDR. This is why the circuit orientation and the observed trend must be reported together. The implementation uses the default publish behavior of PubSubClient, namely MQTT 3.1.1 with QoS 0. Consequently, reception of a sample demonstrates successful best-effort delivery, but it is not a measurement of packet-loss rate or end-to-end reliability.

The block and flow diagrams in Figs. 1 and 2 describe the same sequence that was tested in this checkpoint. The final report must include the corresponding serial-monitor, subscriber, and assembled-circuit captures next to this discussion. These captures should show the actual broker address and readings used during the demonstration, while omitting Wi-Fi passwords and other private credentials.

C. CHECKPOINT 4: Arduino Mega + ESP32 Communication

In this checkpoint, we implemented a small system for simulating microcontroller communication and data transmission via UART and MQTT.

After connecting the Arduino MEGA and ESP32 communication pins, we were able to transmit mock sensor data from the Arduino, receive it successfully on the ESP32, and publish to an MQTT topic, which we subscribed to from a laptop running a Mosquitto client. A voltage divider was required to avoid damaging the ESP, which runs at $3.3V$.

1) Physical Implementation: To enable communication between these two microcontrollers, their serial communication pins must be connected to each other. As their GND reference must also be the same, we connected their ground pins as well.

- Arduino RX1: connected to ESP32 TX2
- Arduino TX1: connected to ESP32 RX2 (with v.divider)
- Arduino GND: connected to ESP32 GND

For the voltage divider, we used three $10k\Omega$ resistors to get $3.33V$ and $1.67V$. We used the first output node, as the ESP32 expects approximately $3.3V$.

The R and T used to name the pins stand for **receive** and **transmit**, which, despite being apparent, took us a minute to notice (sorry).

2) Arduino Code Upload: The Arduino Mega must mock sensor data (or simple messages) and send them via UART to the ESP32. After physically connecting the pins, we can use *Serial.println()* to send a string to the ESP32. However, as the *Serial* object is usually reserved for the USB connection, we used *Serial1* instead. We do this on a loop with a $1000ms$ delay.

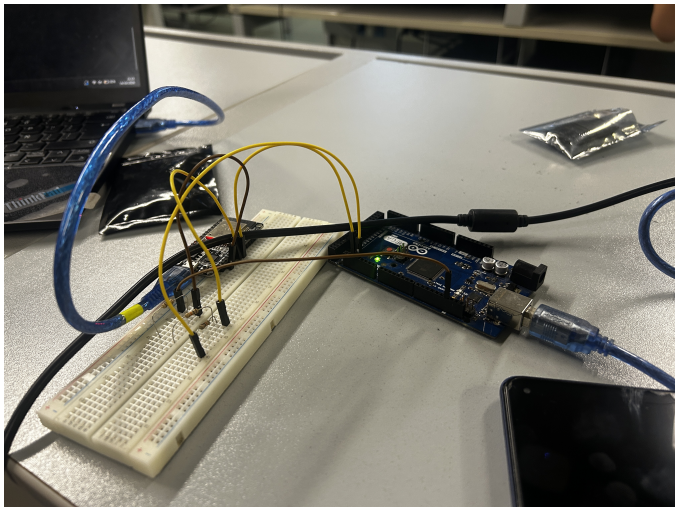


Figure 17. Physical system with a voltage divider to avoid frying the ESP.

3) ESP32 Code Upload: The ESP32 has a more complex task. It must first connect to a WiFi network, set up an MQTT server, receive messages via UART, and publish them to an MQTT topic.

To make sure we didn't run into any issues with the WiFi network, we set up a mobile hotspot using cellular data. The SSID used was **quaso**.

In the setup function, besides initializing both the *Serial* and *Serial2* objects, we connect to the mobile hotspot, set up an MQTT server on port 1883 and then attempt to connect to the MQTT broker.

```
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  WiFi.begin(ssid, password);
  Serial.println(WiFi.localIP());
  mqtt.setServer(mqtt_server, 1883);

  while (!mqtt.connected()) {
    Serial.print("Connecting to MQTT...");

    if (mqtt.connect("esp32cito")) {
      Serial.println("connected!");
    } else {
      Serial.print("failed, state=");
      Serial.println(mqtt.state());
      delay(2000);
    }
  }
}
```

Figure 18. ESP32 code setup

After the setup, the ESP32 reads a string from RX2 and, if valid, publishes it to the MQTT topic. It does this in a loop, while also attempting to reconnect if case the connection with

the broker fails.

4) MQTT Monitoring: After flashing both the Arduino and ESP, and succesfully setting up the config file in checkpoint 1, we ran the Mosquitto broker and, on a separate terminal session, subscribed to the MQTT topic.

IV. Conclusion

Conclusiones y recomendaciones experimentales: por documentar.

References

[1] *CS5055: LAB 3: MQTT & ESP32, Sem 09–Sem 10*, Universidad de Ingeniería y Tecnología, 2026, guía del curso, Sem10_LAB3_MQTT.pdf, pp. 1–11. Acceso institucional; consultada el 14 de octubre de 2026. [Online]. Available: https://utec.instructure.com/courses/24674/files/4935950

[2] *MQTT Version 3.1.1*, OASIS, Oct. 2014, oASIS Standard, 29 de octubre de 2014. Consultado el 14 de octubre de 2026. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[3] *Arduino-ESP32: ADC API*, Espressif Systems, documentación oficial, funciones analogRead y analogReadMilliVolts. Consultada el 14 de octubre de 2026. [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[4] N. O'Leary, "PubSubClient, versión 2.8: código fuente," 2020, pubSubClient.h y PubSubClient.cpp, MQTT_VERSION, publish y loop. Consultados el 14 de octubre de 2026. [Online]. Available: https://github.com/knolleary/pubsubclient/tree/v2.8/src

[5] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[6] D. Thangavel, X. Ma, A. Hoang, and A. Chen, "Performance Evaluation of MQTT and CoAP via a Common Middleware," in *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing*, 2014, pp. 1–6.

[7] O. Georgiou and U. Karapantelakis, "From IoT to Cloud: Applications and Performance of the MQTT Protocol," in *2020 22nd International Conference on Transparent Optical Networks (ICTON)*, 2020, pp. 1–4.

[8] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, "MQTT-S — A publish/subscribe protocol for Wireless Sensor Networks," in *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE)*, 2008, resumen disponible en IBM Research. Consultado el 14 de octubre de 2026. [Online]. Available: https://research.ibm.com/publications/mqtt-s-a-publishsubscribe-protocol-for-wireless-sensor-networks

[9] F. Palmese, A. E. C. Redondi, and M. Cesana, "Adaptive Quality of Service Control for MQTT-SN," *Sensors*, vol. 22, no. 22, p. 8852, 2022.

[10] L. Pacheco and N. Santos, "Delay and Energy Consumption of MQTT over QUIC: An Empirical Characterization Using Commercial-Off-the-Shelf Devices," *Sensors*, vol. 22, no. 10, p. 3694, 2022.